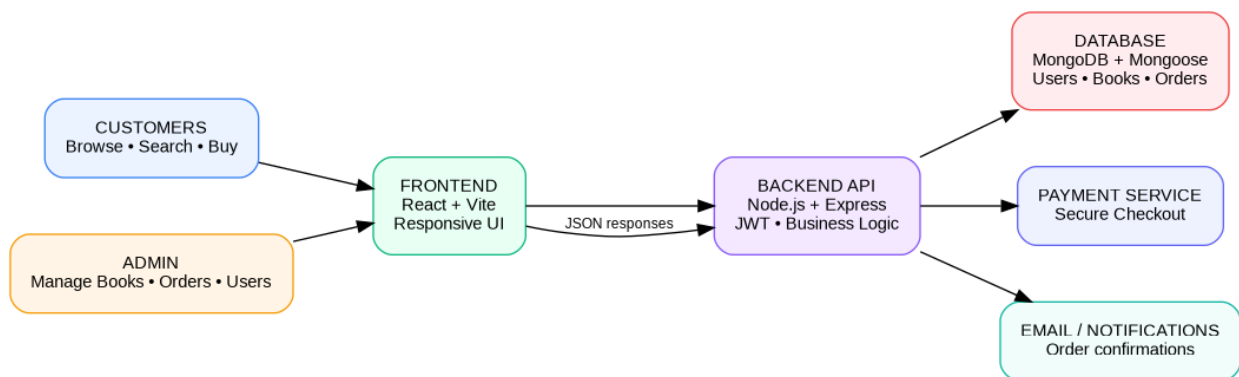


BOOK STORE

ONLINE BOOK SHOPPING PLATFORM

Complete Web Application - Project Documentation



Technology Stack: React.js, Node.js, Express.js, MongoDB, JWT, REST APIs
Prepared for academic project development, implementation, testing, and demonstration.

TABLE OF CONTENTS

- 1. Project Overview
- 2. Technical Architecture
- 3. Entity Relationship Diagram
- 4. Features
- 5. Roles and Responsibilities
- 6. User and Admin Flows
- 7. MVC Architecture
- 8. Client Setup
- 9. Server Setup
- 10. Development Phases
- 11. MongoDB Configuration
- 12. Folder Structure
- 13. Backend Structure
- 14. Database Schemas and Models
- 15. Frontend Structure
- 16. Frontend UI and Logic
- 17. API Modules
- 18. Security and Validation
- 19. Testing Strategy
- 20. Project Execution
- 21. Expected Demo Screens
- 22. Future Enhancements
- 23. Conclusion

1. PROJECT OVERVIEW

The Book Store - Online Book Shopping Platform is a full-stack web application that allows customers to discover, search, compare, purchase, and review books from any device. It replaces manual bookstore operations with a centralized digital system where book information, customer accounts, shopping carts, payments, orders, inventory, and delivery status are managed in one platform.

Customers can create accounts, browse books by category, author, language, price, and rating, view detailed book information, add books to a wishlist or cart, complete checkout, and track orders. Administrators can manage the catalog, stock, prices, users, orders, offers, and reports through a protected dashboard.

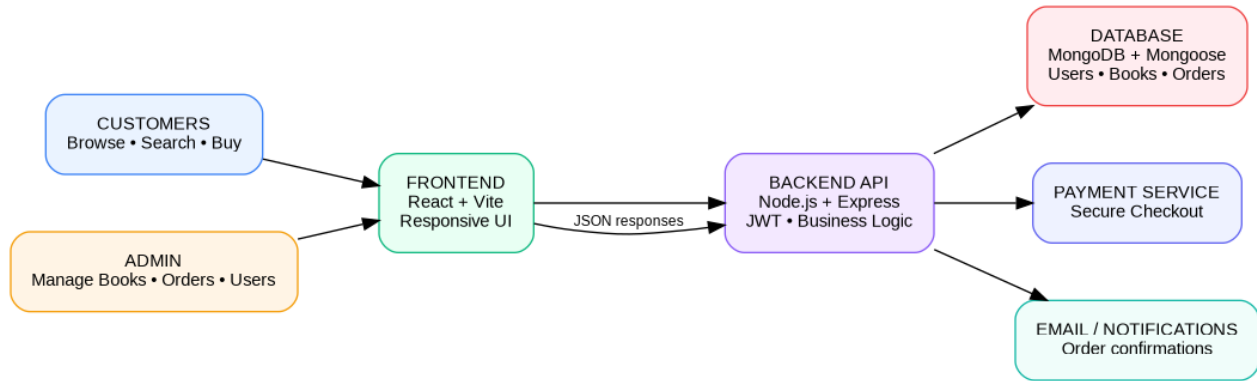
Project Objectives

- Provide a simple and responsive online shopping experience for books.
- Maintain accurate book, stock, customer, and order information.
- Support secure authentication and role-based access for customers and administrators.
- Automate cart calculation, checkout, order confirmation, and order tracking.
- Provide an expandable architecture for payments, recommendations, coupons, and delivery integration.

Technology Stack

Layer	Technology
Frontend	React.js, Vite, HTML5, CSS3, JavaScript, Axios
Backend	Node.js, Express.js, REST APIs
Database	MongoDB with Mongoose
Security	JWT authentication, bcrypt password hashing, role-based authorization
Tools	VS Code, Git, GitHub, Postman, MongoDB Compass / Atlas

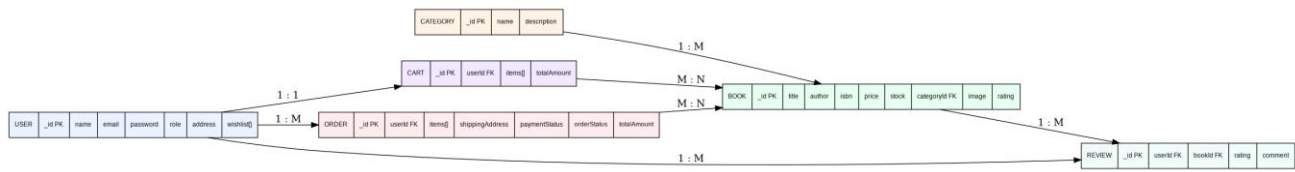
2. TECHNICAL ARCHITECTURE



The application follows a client-server architecture. The React frontend acts as the client and presents catalog, cart, checkout, order, profile, and admin screens. It communicates with the Express backend through REST APIs using JSON. The backend validates requests, applies business rules, authenticates users, manages books and orders, and connects to MongoDB through Mongoose.

External services can be connected for online payments and email notifications. This separation enables the frontend and backend to be developed, tested, deployed, and scaled independently.

3. ENTITY RELATIONSHIP DIAGRAM



The ER diagram represents the main database entities and their relationships. A user owns one cart and may create multiple orders and reviews. Each category contains many books. A cart and an order can contain multiple books, while a book can appear in many carts and orders. Reviews connect customers with the books they have read or purchased.

Main Entities

- User: stores customer or admin account information.
- Book: stores title, author, ISBN, price, stock, image, category, description, and rating.
- Category: organizes books into groups such as Fiction, Technology, Academic, Biography, and Children.
- Cart: stores the customer's selected books and quantities before checkout.
- Order: stores purchased items, delivery address, amount, payment status, and order status.
- Review: stores rating and feedback submitted by customers.

4. FEATURES

1. User Registration and Authentication

- Create a customer account using name, email, phone, and password.
- Secure login and logout using JWT.
- Forgot-password and profile update support can be added.

2. Book Catalog

- Browse latest, popular, discounted, and recommended books.
- Filter by category, author, language, price range, availability, and rating.
- Search by title, author, ISBN, or keyword.

3. Book Details

- Display cover image, title, author, description, price, stock, language, pages, publisher, and rating.
- Show related books and customer reviews.

4. Cart and Wishlist

- Add, remove, and update book quantities.
- Automatically calculate subtotal, discount, delivery fee, and total.
- Save favorite books for later purchase.

5. Checkout and Orders

- Collect shipping address and payment method.
- Create an order and reduce stock after successful checkout.
- Display order history and status: Placed, Confirmed, Packed, Shipped, Delivered, or Cancelled.

6. Admin Management

- Add, edit, delete, and view books and categories.
- Update price, discount, stock, and featured status.
- Manage users, orders, payment status, and delivery status.
- View sales, low-stock, and order reports.

5. ROLES AND RESPONSIBILITIES

Customer

- Register and maintain a personal account.
- Browse and search the book catalog.
- Add books to cart or wishlist.
- Place orders and select delivery details.
- Track orders, cancel eligible orders, and view purchase history.
- Submit ratings and reviews.

Administrator

- Manage books, authors, categories, prices, images, and stock.
- Manage customer accounts and administrator access.
- Review all orders and update order status.
- Monitor payments, cancellations, and refunds.
- View inventory, sales, and customer activity reports.
- Maintain the security and smooth operation of the platform.

Example Scenario

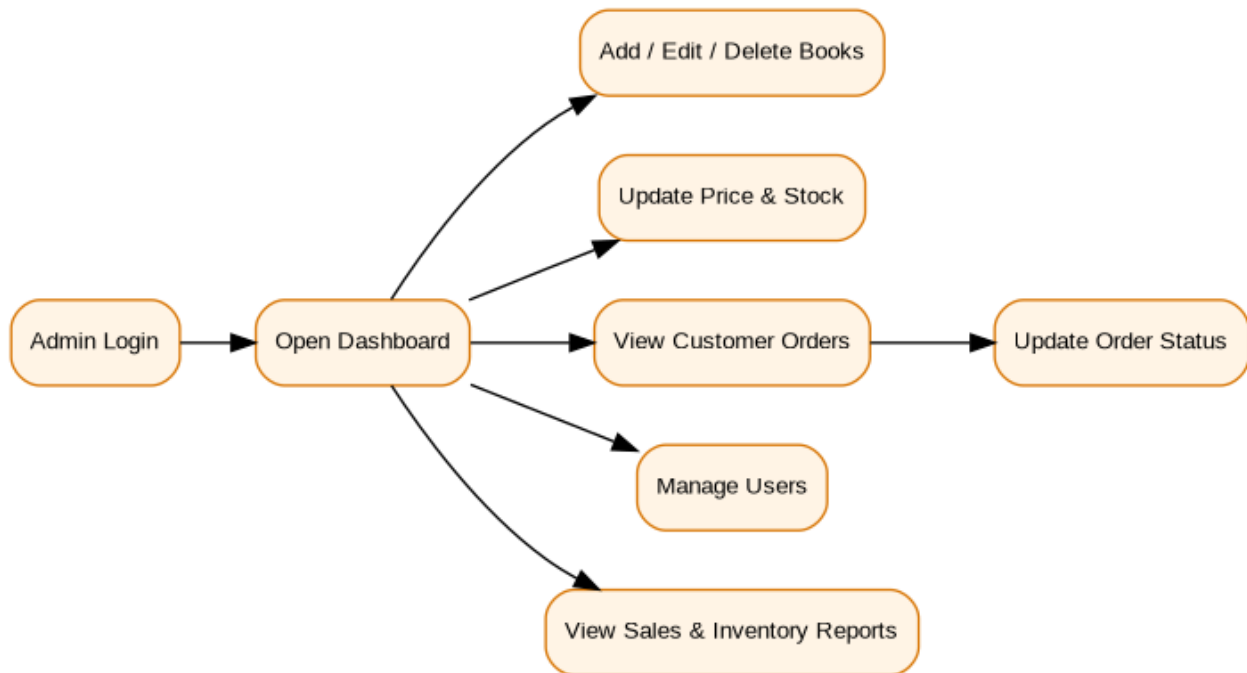
Ananya wants to buy a Java programming book. She creates an account, searches for “Java”, filters books by rating and price, opens a book’s details, adds it to the cart, enters her address, completes payment, and receives an order confirmation. The administrator verifies the order, updates its status from Confirmed to Packed and Shipped, and Ananya tracks the delivery from her account.

6. USER FLOW



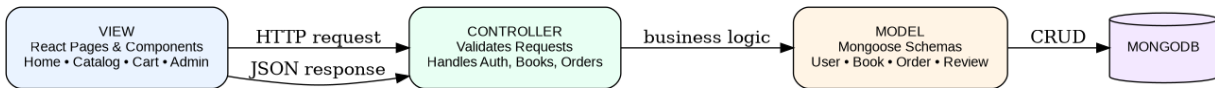
The customer journey begins with account registration or login and continues through discovery, purchase, delivery tracking, and review. At every protected step, the application verifies the JWT token and retrieves current data from the backend.

ADMIN FLOW



The administrator uses a role-protected dashboard to maintain the catalog and process orders. Changes to books, stock, prices, and order statuses are immediately reflected in the customer application.

7. MVC ARCHITECTURE



The project uses the Model-View-Controller design pattern to separate user interface, application logic, and data access.

Model Layer

Mongoose models define and validate User, Book, Category, Cart, Order, and Review data. Models communicate with MongoDB and perform create, read, update, and delete operations.

View Layer

React pages and reusable components display the catalog, book details, cart, checkout, profile, orders, login, registration, and admin dashboard.

Controller Layer

Controllers receive API requests, validate input, apply business rules, call models, and return JSON responses. Examples include registering a user, adding a book, calculating cart totals, placing an order, and updating delivery status.

Advantages

- Clear separation of responsibilities
- Easy maintenance and testing
- Reusable code and components
- Parallel team development
- Simple future expansion

8. CLIENT SETUP (REACT + VITE)

1. Create the project root folder and open it in Visual Studio Code.
2. Create the React frontend:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
```

3. Install common frontend dependencies:

```
npm install axios react-router-dom
# Optional UI packages
npm install lucide-react react-hot-toast
```

4. Start the development server:

```
npm run dev
```

Vite normally starts the frontend at `http://localhost:5173`. The exact URL is displayed in the terminal.

Recommended Frontend Environment Variable

```
VITE_API_URL=http://localhost:5000/api
```

9. SERVER SETUP (NODE + EXPRESS)

5. Create and initialize the backend project:

```
mkdir backend  
cd backend  
npm init -y
```

6. Install the backend dependencies:

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken  
npm install express-validator  
npm install -D nodemon
```

7. Create the basic folders:

```
config/  
controllers/  
middleware/  
models/  
routes/  
utils/  
server.js
```

8. Add scripts in package.json:

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

9. Run the backend server:

```
npm run dev
```

10. DEVELOPMENT PHASES

Phase 1 - Requirement Analysis

Identify customer and admin requirements, major pages, book information, cart rules, order statuses, security needs, and expected reports.

Phase 2 - UI/UX Design

Create responsive layouts for home, catalog, book details, cart, checkout, orders, profile, login, registration, and admin dashboard.

Phase 3 - Database Design

Define User, Book, Category, Cart, Order, and Review schemas with relationships and validation.

Phase 4 - Backend Development

Create authentication, book, category, cart, order, review, and admin APIs. Add controllers, middleware, validation, and error handling.

Phase 5 - Frontend Integration

Connect React pages to APIs using Axios, manage authentication state, implement protected routes, and display live data.

Phase 6 - Testing

Test APIs, validation, authentication, shopping workflows, stock updates, order totals, admin operations, and responsive design.

Phase 7 - Deployment

Deploy frontend and backend separately, configure MongoDB Atlas and environment variables, and verify production API communication.

11. CONFIGURE MONGODB

MongoDB stores application data as flexible JSON-like documents. Mongoose provides schemas, validation, relationships, query methods, and model-based access from Node.js.

Install Mongoose

```
npm install mongoose
```

Database Connection - config/db.js

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected successfully');
  } catch (error) {
    console.error('MongoDB connection failed:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

.env File

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/book_store
JWT_SECRET=replace_with_a_strong_secret
CLIENT_URL=http://localhost:5173
```

12. CREATING FOLDER STRUCTURE

book-store-online-shopping/

```
├── backend/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── utils/
│   ├── .env.example
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── api/
│   │   ├── assets/
│   │   ├── components/
│   │   ├── context/
│   │   ├── pages/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── .env.example
│   ├── package.json
│   └── README.md
```

This separation keeps frontend and backend code independent. It improves readability, team collaboration, testing, deployment, and future scaling.

13. BACKEND FOLDER STRUCTURE

Path	Purpose
config/db.js	Creates the MongoDB connection.
middleware/auth.js	Verifies JWT tokens and attaches the logged-in user.
middleware/admin.js	Allows only administrator accounts.
models/	Contains User, Book, Category, Cart, Order, and Review schemas.
controllers/	Contains business logic for every module.
routes/	Defines REST API endpoints.
utils/	Contains helper functions such as token generation and email utilities.
server.js	Initializes Express, middleware, database connection, and routes.

Main Route Files

- authRoutes.js - registration, login, current user
- bookRoutes.js - catalog and admin book CRUD
- categoryRoutes.js - category management
- cartRoutes.js - add, update, remove, and retrieve cart items
- orderRoutes.js - checkout, order history, cancellation, and status updates
- reviewRoutes.js - ratings and comments
- userRoutes.js - profile and admin user management

14. DATABASE SCHEMAS AND MODELS

Book Model - Important Fields

```
title: String
author: String
isbn: String (unique)
description: String
price: Number
discountPrice: Number
stock: Number
category: ObjectId -> Category
coverImage: String
language: String
pages: Number
publisher: String
averageRating: Number
isFeatured: Boolean
```

Order Model - Important Fields

```
user: ObjectId -> User
items: [{ book, title, price, quantity }]
shippingAddress: Object
paymentMethod: String
paymentStatus: Pending | Paid | Failed | Refunded
orderStatus: Placed | Confirmed | Packed | Shipped | Delivered | Cancelled
subtotal: Number
discount: Number
deliveryFee: Number
totalAmount: Number
createdAt / updatedAt
```

Data Validation Rules

- Email and ISBN must be unique.
- Price and stock cannot be negative.
- Quantity must be at least one and cannot exceed available stock.
- Rating must be between 1 and 5.
- Only administrators can create, update, or delete books.
- Only the order owner or an administrator can view an order.

15. FRONTEND FOLDER STRUCTURE

```
frontend/src/  
├── api/  
│   └── axios.js  
├── assets/  
│   └── images/  
├── components/  
│   ├── Navbar.jsx  
│   ├── Footer.jsx  
│   ├── BookCard.jsx  
│   ├── SearchBar.jsx  
│   ├── ProtectedRoute.jsx  
│   └── AdminRoute.jsx  
├── context/  
│   ├── AuthContext.jsx  
│   └── CartContext.jsx  
├── pages/  
│   ├── Home.jsx  
│   ├── Books.jsx  
│   ├── BookDetails.jsx  
│   ├── Cart.jsx  
│   ├── Checkout.jsx  
│   ├── Orders.jsx  
│   ├── Login.jsx  
│   ├── Register.jsx  
│   └── admin/  
│       ├── Dashboard.jsx  
│       ├── ManageBooks.jsx  
│       └── ManageOrders.jsx  
├── App.jsx  
├── main.jsx  
└── index.css
```

Reusable components ensure consistent design. Context modules manage authentication and cart state globally, while page components represent complete screens.

16. DESIGN UI COMPONENTS

Navbar

Displays the logo, search, categories, cart count, login/profile options, order link, and admin dashboard link when applicable.

BookCard

Shows the book cover, title, author, rating, price, discount, stock status, and add-to-cart action.

Filters and Search

Allow customers to find books quickly by keyword, category, price, language, author, and rating.

Cart Summary

Shows selected items, quantities, subtotal, discounts, delivery fee, and payable total.

Admin Tables

Display books, users, and orders with actions such as view, edit, delete, and status update.

Responsive Layout

Adapts navigation, cards, forms, and tables for desktop, tablet, and mobile screens.

IMPLEMENT FRONTEND LOGIC

Axios sends requests to backend APIs. AuthContext stores the user and token, CartContext stores cart information, ProtectedRoute prevents unauthenticated access, and AdminRoute blocks non-admin users. React state and effects load books, apply filters, update quantities, submit checkout data, and refresh order status.

17. API MODULES

Method & Endpoint	Access	Purpose
POST /api/auth/register	Public	Create customer account
POST /api/auth/login	Public	Authenticate and return JWT
GET /api/books	Public	List and filter books
GET /api/books/:id	Public	Get complete book details
POST /api/books	Admin	Add a new book
PUT /api/books/:id	Admin	Update book details or stock
DELETE /api/books/:id	Admin	Delete a book
GET /api/cart	Customer	Get current cart
POST /api/cart/items	Customer	Add book to cart
PUT /api/cart/items/:bookId	Customer	Update quantity
POST /api/orders	Customer	Place an order
GET /api/orders/my-orders	Customer	View order history
GET /api/orders	Admin	View all orders
PATCH /api/orders/:id/status	Admin	Update order status
POST /api/reviews/:bookId	Customer	Add rating and review

18. SECURITY AND VALIDATION

Password Security

Passwords are never stored as plain text. bcrypt creates a secure password hash before saving the user.

JWT Authentication

After successful login, the server issues a signed token. Protected routes verify the token before processing requests.

Role-Based Authorization

Administrator middleware verifies the user role before allowing catalog, user, or order management actions.

Input Validation

Server-side validation checks required fields, email format, price, stock, quantity, rating, and valid MongoDB IDs.

Secure Configuration

Secrets and database URIs are stored in environment variables and are excluded from Git.

Error Handling

Central error middleware returns meaningful status codes without exposing internal server details.

Business Rule Protection

The server rechecks stock and calculates totals during checkout instead of trusting values sent by the browser.

19. TESTING STRATEGY

Functional Testing

- Registration, login, logout, and protected routes
- Book search, sorting, filtering, and pagination
- Cart add, remove, quantity update, and total calculation
- Checkout, stock reduction, and order creation
- Order history, cancellation rules, and status tracking
- Admin book CRUD and order status updates
- Review submission and average rating calculation

API Testing with Postman

Create a Postman collection for every endpoint. Test valid requests, missing fields, invalid tokens, unauthorized roles, unavailable stock, duplicate email or ISBN, and non-existing resources.

UI and Responsive Testing

Verify layouts and interactions on common desktop, tablet, and mobile sizes. Check loading states, empty states, validation messages, navigation, forms, and long book titles.

Performance and Reliability

- Use pagination for large catalogs.
- Add database indexes for email, ISBN, category, title, and author.
- Avoid returning password fields.
- Handle database or network failures gracefully.

20. PROJECT EXECUTION

Step 1 - Start MongoDB

Start the local MongoDB service or configure a MongoDB Atlas connection string.

Step 2 - Start Backend

```
cd backend
npm install
# create .env
npm run dev
```

Step 3 - Start Frontend

```
cd frontend
npm install
# create .env with VITE_API_URL
npm run dev
```

Step 4 - Open Application

Open the Vite URL shown in the terminal, usually `http://localhost:5173`. The backend normally runs at `http://localhost:5000`.

Production Deployment

- Deploy the frontend to Vercel or Netlify.
- Deploy the backend to Render, Railway, or another Node.js hosting platform.
- Use MongoDB Atlas for the production database.
- Set `CLIENT_URL`, `MONGO_URI`, `JWT_SECRET`, and other environment variables on the hosting platform.
- Update `VITE_API_URL` to the deployed backend API URL.

21. EXPECTED DEMO SCREENS

Landing / Home Page

Hero section, search bar, categories, featured books, new arrivals, discounts, and customer benefits.

Registration and Login

Forms for account creation and secure authentication with validation messages.

Book Catalog

Grid of books with filters, sorting, search, pagination, prices, ratings, and stock status.

Book Details

Complete book information, cover, reviews, related books, quantity, add-to-cart, and wishlist actions.

Cart

Selected books, quantity controls, remove action, pricing summary, and checkout button.

Checkout

Shipping address, order summary, payment method, and place-order confirmation.

My Orders

Order history, status timeline, amount, items, delivery address, and cancellation option.

Admin Dashboard

Summary cards for books, users, orders, revenue, and low-stock products.

Manage Books

Table and form for adding, editing, deleting, searching, and updating stock.

Manage Orders

All orders with customer details, payment status, amount, and delivery status controls.

22. FUTURE ENHANCEMENTS

- Online payment gateway integration such as Razorpay or Stripe.
- Coupon codes, seasonal offers, gift cards, and loyalty points.
- AI-based book recommendations using browsing and purchase history.
- Real-time inventory and delivery notifications.
- Seller or publisher portal for multi-vendor catalog management.
- E-book purchase and secure digital reading.
- Advanced analytics for sales, customers, categories, and stock forecasting.
- Chatbot for book discovery and order support.
- Multi-language and accessibility support.

23. CONCLUSION

The Book Store - Online Book Shopping Platform provides a complete digital solution for catalog browsing, secure shopping, order management, inventory control, and administration. Its React, Express, MongoDB, and JWT architecture is modular, maintainable, and suitable for academic demonstration as well as future real-world expansion. The project demonstrates frontend development, REST API design, database modeling, authentication, authorization, validation, testing, and deployment in one integrated application.

Demo link:

https://drive.google.com/file/d/1TscWLJykz9V9dijj_UTbySsldAcJp1_F/view?usp=sharing

End of Document